

P3639R0

The `_BitInt` Debate

Published Proposal, 2025-02-20

This version:

<https://eisenwave.github.io/cpp-proposals/bitint.html>

Author:

[Jan Schultke](#)

Audience:

SG6, EWG, LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:

[eisenwave/cpp-proposals](#)

Abstract

An N-bit integer type similar to C's `_BitInt` would provide utility and give ABI compatibility to C++. However, should this be a fundamental type or a library type?

Table of Contents

1 Introduction

2 Motivation

2.1 Computation beyond 64 bits

2.2 C-ABI compatibility

3 Scope

4 Design

4.1 Introduction

4.2 Pro-fundamental: Full compatibility with C constructs

4.3 Pro-fundamental: `unsigned _BitInt`

4.4 Pro-fundamental: `_BitInt` and `class bit_int` don't coexist nicely

4.5 Pro-fundamental: `_BitInt` needs to exist in C anyway

4.6 Pro-fundamental: `_BitInt` is fast to compile

4.7 Pro-fundamental: A pure library implementation is only a temporary solution

4.8 Pro-fundamental: `_BitInt` offers overload resolution flexibility

4.9 Pro-fundamental: `_BitInt` could have special deduction powers

- 4.10 Pro-library: `_BitInt` is not portable, but `class bit_int` can be
- 4.11 Pro-library: `_BitInt` inherits many ill-conceived integer features
- 4.12 Pro-library: `class bit_int` is easier to implement
- 4.13 Pro-library: We'll likely get a portable `class bit_int` faster
- 4.14 Pro-library: `class bit_int` is easier to teach
- 4.15 Pro-library: `class bit_int` has much less wording impact and has fewer gotchas

References

Informative References

§ 1. Introduction

[N2763] introduced the `_BitInt` set of types to the C23 standard, and [N2775] further enhanced this feature with literal suffixes. For example, this feature may be used as follows:

```
// 8-bit unsigned integer initialized with value 255.
// The literal suffix wb is unnecessary in this case.
unsigned _BitInt(8) x = 0xFFwb;
```

In short, the behavior of these *bit-precise integers* is as follows:

- No integer promotion to `int` takes place.
- Mixed-signedness comparisons, implicit conversions, and other permissive feature are supported.
- They have lower conversion rank than standard integers, so an operation between `_BitInt(8)` and `int` yields `int`, as does an operation with `_BitInt(N)` where `N` is the width of `int`. They only have greater conversion rank when their width is greater.

These semantics make it clear that bit-precise integers are complementary to the standard integers, not a replacement, and not an attempt at fixing all the semantics that users consider overly permissive about standard integers.

I propose that C++ should also have an N-bit integer type, possibly as a library type. This is similarly an attempt at enhancing the language, not at replacing the standard integers. The C++ types should be ABI-compatible with `_BitInt` in C. Unfortunately, one question blocks any concrete steps:

Should C++ have a fundamental `_BitInt` type (possibly exposed via alias template), or should it have a library type (class template)?

The sole purpose of this proposal is to reach consensus on a direction.

§ 2. Motivation

§ 2.1. Computation beyond 64 bits

One of my previous proposals ([P3140R0]) provides a sea of motivation for 128-bit integers alone. [P3140R0] consistently got the following feedback (from LEWGI and outside):

- Yes, 128-bit computation is useful!

- What about more than 128 bits?

Some use cases extend beyond 128 bits, such as 4096-bit (or more) computation for RSA and other cryptographic algorithms.

The original [N2763] C proposal and C++11 proposals which previously proposed this feature ([N1692] and [N1744]) contain further motivation. [N4038] also proposed a "big integer", although a dynamically-sized one. [P1889R1] (TS) also contained a `wide_integer` class template.

N-bit integers seems to have been suggested and worked on *many* times in the past decades; they simply weren't prioritized by the authors and by WG21 so as to make it into the standard. I strongly doubt that the usefulness of integer arithmetic beyond 64 bits is contentious, so it won't be discussed further here.

§ 2.2. C-ABI compatibility

C++ currently has no portable way to call a C function such as:

```
_BitInt(N) plus(_BitInt(N) x, _BitInt(N) y); // for any N
```

We need to call C functions that use `_BitInt` somehow. This could be accomplished either with a fundamental type that has identical ABI, or with a class type in the style of `std::complex` or `std::atomic` that has identical ABI despite being a class type.

This compatibility problem is not a hypothetical concern either; it is an urgent problem. There are already targets with `_BitInt`, supported by major compilers, and used by C developers:

Compiler	BITINT_MAXWIDTH	Targets	Languages
clang 16+	8388608	all	C & C++
GCC 14+	65535	64-bit only	C
MSVC 19.38	✗	✗	✗

§ 3. Scope

I only propose *fixed-width, N-bit, signed and unsigned* integers.

I do not propose dynamically sized, "infinite precision" integers. Such integers

- would not be motivated by C compatibility,
- would require discussing allocator-awareness, small object optimizations, and other complex issues, and
- they obviously should be done as library types anyway.

Therefore, they are outside the scope of this proposal.

§ 4. Design

§ 4.1. Introduction

At this point, there are two plausible implementations:

Fundamental type	Library type
<pre>template <size_t N> using bit_int_t = _BitInt(N); template <size_t N> using bit_uint_t = unsigned _BitInt(N);</pre>	<pre>template <size_t N> class bit_int { private: _BitInt(N) _M_value; public: // constructors, operator overloads, ... }; // analogous for bit_uint wrapping unsigned _BitInt</pre>

NOTE: These implementations are already valid C++ code when using the Clang `_BitInt` compiler extension. Without such an extension, a library type can still be implemented in software, similar to [Boost.Multiprecision's `cpp\_int`](#).

In terms of ABI and performance (after inlining), these two approaches should yield the same results.

It may also be possible to allow the `bit_int_t` alias template to alias a class template, but this inevitably results in implementation divergence. For example, during overload resolution, `bit_int_t` in some implementations would have user-defined conversion sequences and standard conversion sequences in others. This is a minefield for users; it really needs to *always* be fundamental or *always* a class.

Also, we could expose the `_BitInt` keyword directly, but this would contradict all previous design:

- `_Atomic` in C is `std::atomic` in C++
- `_Float128` in C is `std::float128_t` in C++
- `complex` in C is `std::complex` in C++
- ...

However, it would be appropriate to define `_BitInt` and `_BitUInt` compatibility macros, similar to the `_Atomic(T)` C++ macro for `std::atomic<T>`:

```
#ifdef __cplusplus
    #define _BitInt(...) std::bit_int<__VA_ARGS__>
    #define _BitUInt(...) std::bit_uint<__VA_ARGS__>
#elifdef __STDC__
    // _BitInt is a keyword, so define only _BitUInt
    #define _BitUInt(...) typedef(unsigned _BitInt(...))
#else
    #error "owo what's this language?"
#endif
```

With these secondary concerns out of the way, we can discuss the big question: fundamental type or library type?

Disclaimer: the author has no strong preference.

§ 4.2. Pro-fundamental: Full compatibility with C constructs

C permits the use of `_BitInt` in situations where a C++ library type couldn't be used, including `switch`s and bit-fields:

```
// OK: using an unsigned _BitInt(1) in a switch
switch (0uwb) // ...
```

```
struct S {
    _BitInt(32) x : 10; // OK, bit-field
};
```

A C++ library type would be less powerful by default, and `S` could not be portably used from C++.

✗ Counterpoint: Conditionally-supported exemptions could be made which allow the use of `bit_int` in those cases, despite being a class type. If `bit_int` is just a wrapper for `_BitInt`, that may be possible, but obviously not if it's purely library-implemented.

§ 4.3. Pro-fundamental: unsigned _BitInt

If there was a `class bit_int` and `_BitInt` compatibility macro for interop with C, then `unsigned _BitInt` obviously wouldn't work because it expands to `unsigned bit_int`.

Unless we make `unsigned class bit_int` valid (which would be highly unusual for C++), both C and C++ uses are forced to use the `_BitUInt` macro for unsigned types. This seems like needless bullying of C users, who can use `unsigned _BitInt` just fine.

With a `_BitInt` fundamental type, we could simply define `_BitInt` and `unsigned _BitInt` in C++, and offer users `bit_int_t` and `std::bit_uint_t` alias templates for a nicer spelling.

NOTE: This problem does not arise for `_Atomic` because we can simply say `_Atomic(unsigned)`, which is valid in C with no macros, and expands to `std::atomic<unsigned>` in C++. It also doesn't arise for `_Float128` or `complex` because there are no unsigned floating-point types; the issue is entirely limited to `_BitInt`.

§ 4.4. Pro-fundamental: _BitInt and class bit_int don't coexist nicely

When `bit_int` is a class template, the idiomatic implementation (see above) is to wrap a `_BitInt` in that template. We can expect that in some implementations, both `class bit_int` and `_BitInt` exist. Clang already provides `_BitInt` in C++ mode.

This actually leads to some major problems, such as:

```
// api.h (C/C++ interoperable header)
_BitInt(32) gimme_bit_int(void);
```

With the `_BitInt` compatibility macro mentioned previously, include order matters:

```
#include <stdint> // OK: contains _BitInt compatibility macro
#include "api.h"  // OK: gimme_bit_int returns class bit_int
```

OR

```
#include "api.h" // OMG: gimme_bit_int returns a non-standard compiler extension
#include <stdint> // OMG: compatibility macro now makes it impossible to call gimme_bit_int
```

There is no obvious way to fix this problem:

- Predefining `_BitInt` in C++ to mean `class bit_int` would not be viable because some users would like a way to access the underlying type in the compiler, not just the wrapper class.
- We could define yet another `_CppBitInt` macro that expands to either `_BitInt` in C, or `class bit_int` in C++, but this is much more annoying for C users.

NOTE: This issue arises less for `_Atomic(T)` because `std::atomic<T>` is usually a wrapper class for `T`. There's no urgent need for `_Atomic` as a builtin type to exist.

On the other hand, if `bit_int_t` is merely an alias, the compiler can simply never define the compatibility macro, and `_BitInt` has the same meaning everywhere, regardless of include order.

✗ Counterpoint: If `<stdint.h>` defined the compatibility macro, the user could have fixed this problem by including that header in `api.h`.

§ 4.5. Pro-fundamental: `_BitInt` needs to exist in C anyway

Any postmodern C++ compiler is also a C compiler. While GCC has a separate frontend for C and C++, the general machinery behind `_BitInt` exists already, assuming that C23 is supported. Therefore, we are almost "throwing away" what's there already by not guaranteeing that `_BitInt` is a library type.

Furthermore, a library type would deviate at least somewhat from the semantics of `_BitInt` in C, making code between the languages less interchangeable.

✗ Counterpoint: See § 4.10 Pro-library: `_BitInt` is not portable, but `class bit_int` can be. The minimum `_BitInt` support in C23 is highly limited. We should think about the features we want to provide to C++ developers, and the guarantees of the C feature are insufficient for that. Furthermore, it's unclear when (or if ever) MSVC will support `_BitInt`.

§ 4.6. Pro-fundamental: `_BitInt` is fast to compile

A fundamental type would incur very little cost during constant evaluation, and would not require any template machinery. This may substantially improve compilation speed as compared to a library type.

✗ Counterpoint: Similar to `std::array`, if `bit_int` is merely a wrapper for a compiler extension type, this cost may be relatively low.

§ 4.7. Pro-fundamental: A pure library implementation is only a temporary solution

With current compiler technology, certain optimizations are only realistic for fundamental types. For example, an integer division with constant divisor like `x / 10` can be turned into a fixed-point multiplication with 10^{-1} :

```
; unsigned div10(unsigned x) { return x / 10; }
div10(unsigned int):
    mov     ecx, edi
    mov     eax, 3435973837
    imul    rax, rcx          ; multiplication with inverse of 10, shifted 35 bits to left
    shr     rax, 35           ; right-shift by 35 bits to correct
    ret
```

NOTE: Compilers transform like this because integer division is one of the most expensive arithmetic operations; it may take over 100 cycles on some architectures. `operator/` is also tremendously more complicated to implement than `operator*` in multi-precision libraries.

That optimization is possible for fundamental types, but a library-implemented division with `bit_int<4096>` could result in hundreds or thousands of IR instructions being emitted, which the compiler cannot realistically interpret as an integer division.

A pure library implementation is not transparent to the compiler in the same way that `_BitInt` is, and prohibits many such optimizations. **If we know already that such an implementation is suboptimal and should be replaced eventually, why make it an option in the first place?** In other words, if `_BitInt` *must* be present for acceptable quality of implementation, any portability/ease-of-implementation argument in favor of `class bit_int` is refuted.

At best, `class bit_int` would be more portable, but may optimize dramatically worse when not implemented via intrinsics. Furthermore, implementers are not interested in having *both* a library implementation and builtin implementation. If one obsoletes the other, that is simply twice the work.

✗ Counterpoint: Not every use case requires these optimizations, and not every use case requires integer division. Multi-precision libraries have existed long before `_BitInt`, and many clever code transformations can be performed by the user manually.

§ 4.8. Pro-fundamental: `_BitInt` offers overload resolution flexibility

With a library type, it may be difficult to achieve certain behavior in overload resolution. For example, people have expressed interest in writing overload sets like:

```
void f(bit_int_t<32>); // alias for _BitInt(32)
void f(bit_int_t<16>);
// ...
```

When called with `bit_int_t<20>`, this could prefer to call `f(bit_int_t<32>)` because the conversion is lossless. With class types, this is difficult/impossible to achieve because user-defined conversion sequences generally rank the same.

✗ Counterpoint: While useful, this idea is novel, and it's not obvious that we want/expect this. For example, calling a function with `long` does not prefer lossless conversions to `long long` over lossy conversions to `int`. `bit_int_t` is all about being explicit with integer widths, and highly flexible implicit conversions "don't fit the theme".

§ 4.9. Pro-fundamental: `_BitInt` could have special deduction powers

Multiple committee members have expressed interest in the following construct:

```
template <size_t N>
void foo(bit_int_t<N>);

foo(0); // OK, calls foo<32> on most platforms
```

This is obviously impossible with a class template, but could be permitted with special deduction rules for `bit_int_t<N>` when passing a standard integer. C++ users may perceive this as a case that should *intuitively* work, even if it usually doesn't (similar issues with `optional<T>`).

Such deduction is also quite useful because

- `foo` can accept any signed standard integer and any bit-precise integer,
- there are fewer instantiations of `foo` if we convert `int` to `bit_int_t<32>` at the call site, and
- it gives `foo` quick access to the width, `N`, even when `int` was passed.

✗ Counterpoint: The problem is much more general, and the usual workaround is to rely on CTAD. For example, if `bit_int` was a class template, we could write:

```
bit_int(int) -> bit_int<32>; // deduction guide
// ...
foo(bit_int(0)); // OK, passes bit_int<32> to foo, and foo deduces as usual
```

Similar workarounds are common for `std::optional`, `std::span`, etc. Rather than carving out a special case in the deduction rules, it would be desirable to solve this in general, which is proposed in [P2998R0]. With that proposal, the motivating example would also work for `class bit_int`.

§ 4.10. Pro-library: `_BitInt` is not portable, but `class bit_int` can be

One of the greatest downsides of `_BitInt` is that it's effectively an optional type. This stems from the fact that only widths of at least `LLONG_WIDTH` are guaranteed.

The purpose of `bit_int` is primarily to provide portable multi-precision to C++ developers. If we are not guaranteed a `BITINT_MAXWIDTH` more than 64 bits, then `_BitInt` miserably fails this goal.

It is also unlikely that this limit could be bumped on the C++ side. Back when I proposed [P3104R0] at LEWG, and in prior discussion, it was often put into question whether 128-bit types should be mandatory. There were major implementer concerns regarding this, and I was advised to make the type freestanding-optional. A fundamental `_BitInt(128)` suffers from the same issue and would receive the same criticism.

On the contrary, if `bit_int` is allowed to be library-implemented, these concerns vanish, and the type can be freestanding-mandatory with a very high maximum width. After all, the library implementation is basically a `std::array` with some operator overloads.

✗ Counterpoint: This concern is mostly theoretical. There's little motivation for an implementation to only provide `_BitInt(64)`. In practice, GCC and Clang already provide `_BitInt` with very large width (§ 2.2 C-ABI compatibility), although GCC does not yet make this type available on 32-bit targets.

§ 4.11. Pro-library: `_BitInt` inherits many ill-conceived integer features

`_BitInt` in C inherits many ill-conceived, extremely permissive features from other standard integers. For example, with `_BitInt`,

- mixed-signedness implicit conversions can take place, such as in `_BitInt(5) + unsigned`,
- mixed-signedness comparisons are also possible, and
- narrowing conversions from e.g. `_BitInt(32)` to `_BitInt(8)` are permitted.

A particular pitfall is that `unsigned _BitInt(N)` with `N <= INT_WIDTH` has lower conversion rank than `int`, so `x + 1` changes signedness, and `x * int(/* large value */) may overflow despite x being an unsigned bit-precise integer.`

With C++26's focus on safety, it is highly questionable whether this behavior should be carried over into C++. A library type could simply start a safer design from scratch, and only provide desirable overloads for `operator+` etc.

✗ Counterpoint: Such misuses could be inherited from C, but diagnosed with Profiles, assuming those will be in C++29. Furthermore, `_BitInt` in C++ could be restricted so that some of this behavior is disallowed (ill-formed).

§ 4.12. Pro-library: `class bit_int` is easier to implement

There are already numerous implementations of multi-precision integers for C++, such as Boost.Multiprecision. These have been tried and tested over many years, and can be integrated into the standard library. If a pure library implementation is permitted, the burden is lowered from having to implement N-bit operation codegen in a compiler, to implementing the underlying `__builtin_add_overflow` et al. intrinsics used in the standard library (or other library), for a given architecture.

Standardizing `_BitInt` as a fundamental type also forces the implementation to set an ABI for this type in stone. Doing so is of much greater consequence than deciding on an ABI for `class bit_int`, where, if push comes to shove, an ABI break is more plausible than for a fundamental C type.

Even the frontend implementation requires some effort, such as

- parsing `_BitInt(N)` and `unsigned _BitInt(N)` with appropriate diagnostics,
- template argument deduction of `N` from `_BitInt(N)`,
- new rules for overload resolution, conversion ranks, etc.,
- ...

✗ Counterpoint: The implementation effort is non-trivial regardless, especially if the implementation has to be of high quality/performance. Even a library implementation needs to exploit architectural knowledge in the form of intrinsics or inline assembly.

§ 4.13. Pro-library: We'll likely get a portable `class bit_int` faster

At this point, it is unclear when (or if ever) Microsoft will implement `_BitInt`. It is not even clear whether 128-bit integer integers are planned, let alone 4096-bit support or higher. Barely any C23 core features are supported at this time. On the contrary, the MSVC STL is largely on par with other standard libraries regarding postmodern C++ support.

Ultimately, we want C++ developers to receive a portable feature soon, and requiring a fundamental type gets us this feature much later or never, depending on how MSVC and possible future compilers prioritize `_BitInt`.

Note that a pure library implementation of `class bit_int` can be replaced with a wrapper for a built-in `_BitInt` type at a later date, convenient for implementations. The option to have a pure library implementation gets the foot in the door.

✗ **Counterpoint:** This point becomes less significant once all compilers support `_BitInt(N)` with large `N`. Maybe this makes for a less timeless design.

§ 4.14. Pro-library: `class bit_int` is easier to teach

Generally speaking, new fundamental types introduce much more complexity for C++ users than library types. For example, `_BitInt` is not subject to integer promotion, but subject to integer conversion. Assuming we carry this over into C++, this has surprising consequences such as:

```
// Currently valid overload set which covers all existing signed standard integers.
void awoo(int);
void awoo(long);
void awoo(long long);
```

`awoo` can be called with any existing signed standard integer, but the overload set is insufficient for `_BitInt` because none of these functions are a best match.

On the contrary, With a `class bit_int`, this behavior would be glaringly obvious, and such a class type would presumably not have a user-defined conversion function to integers anyway. In practice, users can simply look at cppreference and find all the constructors, operator overloads, and quickly understand how `bit_int` works.

✗ **Counterpoint:** This point is somewhat subjective and speculative. Either way, there will be new sets of types in the language, and new users will have to learn about them if they want to use them.

§ 4.15. Pro-library: `class bit_int` has much less wording impact and has fewer gotchas

Merely introducing a new class type to the language has no impact on the core wording whatsoever, and is absolutely guaranteed not to introduce wording bugs and subtle ABI breaks.

To name one potential issue, `IOTA-DIFF-T(W)` for `W = long long` is defined as ([\[range.iota.view\]](#)):

a signed integer type of width greater than the width of `W` if such a type exists.

If we consider `_BitInt` to be a signed integer type (why wouldn't it be?), this breaks ABI because we are redefining a `difference_type`. This bug is very similar to `std::intmax_t`, which has prevented the implementation from providing extended 128-bit integers without an ABI break.

This `iota_view` issue can be mitigated by requesting that only *standard* signed integers are considered in that bullet, not *all* signed integers. However, it demonstrates that adding an entirely new set of fundamental integer types comes with great wording impact.

Is resolving all these problems a good use of committee time? Adding a new class to the standard library *just works*.

✗ **Counterpoint:** This is putting the cart before the horse. While committee time is a valuable resource, wording impact should not dictate design.

§ References

§ Informative References

[N1692]

M.J. Kronenburg. *A Proposal to add the Infinite Precision Integer to the C++ Standard Library*. 1 July 2004. URL: <https://wg21.link/n1692>

[N1744]

Michiel Salters. *Big Integer Library Proposal for C++0x*. 13 January 2005. URL: <https://wg21.link/n1744>

[N2763]

Aaron Ballman; et al. *Adding a Fundamental Type for N-bit integers*. URL: <https://open-std.org/JTC1/SC22/WG14/www/docs/n2763.pdf>

[N2775]

Aaron Ballman; Melanie Blower. *Literal suffixes for bit-precise integers*. URL: <https://open-std.org/JTC1/SC22/WG14/www/docs/n2763.pdf>

[N4038]

Pete Becker. *Proposal for Unbounded-Precision Integer Types*. 23 May 2014. URL: <https://wg21.link/n4038>

[P1889R1]

Alexander Zaitsev, Antony Polukhin. *C++ Numerics Work In Progress*. 27 December 2019. URL: <https://wg21.link/p1889r1>

[P2998R0]

James Touton. *CTAD for function parameter types*. 15 October 2024. URL: <https://wg21.link/p2998r0>

[P3104R0]

Jan Schultke. *Bit permutations*. 26 January 2024. URL: <https://wg21.link/p3104r0>

[P3140R0]

Jan Schultke. *std::int_least128_t*. 14 February 2024. URL: <https://wg21.link/p3140r0>